Lab 06. Đánh giá độ chính xác và hiệu năng Tiny CNN V5 INT8 trên 995 ảnh MRI bằng ARM + FPGA

Step 1. Mục tiêu và tập dữ liệu kiểm thử

Lab 06 đánh giá mô hình Tiny CNN V5 INT8 theo hai phương án ARM thuần và ARM + FPGA trên **toàn bộ 995 ảnh MRI Test**, thay cho thử nghiệm một ảnh ở Lab 05. Mục tiêu là tính Accuracy, Precision, Recall, F1-score, Confusion Matrix, so sánh thời gian suy luận và kiểm tra đầu ra INT8 của hai phương án có trùng khớp hay không.

Nhãn MRI	Số ảnh Test
Glioma	253
Meningioma	303
No tumor	139
Pituitary	300
Tổng cộng	**995**

Các ảnh được nén vào **lab06_test_995.zip**, tải lên thư mục Jupyter **Lab_06**, sau đó giải nén và chuẩn hóa tên đường dẫn từ Windows sang Linux. Cấu trúc cuối cùng là test_data/glioma, test_data/meningioma, test_data/no_tumor và test_data/pituitary, đủ đúng số ảnh từng lớp.

Step 2. Chương trình chuẩn bị dữ liệu Test

Ô code thứ nhất có nhiệm vụ kiểm tra file ZIP, sửa tên file có dấu gạch chéo ngược, chỉ giải nén ảnh còn thiếu và kiểm tra chính xác số lượng ảnh từng lớp. Khi dữ liệu đã chuẩn bị xong, có thể chạy lại ô code mà không giải nén trùng lặp.

```
# CELL 1 - PREPARE MRI TEST DATASET
from pathlib import Path
from zipfile import ZipFile
import shutil

ZIP_PATH = Path("lab06_test_995.zip")
DATASET_ROOT = Path("test_data")
EXPECTED = {
    "glioma": 253,
    "meningioma": 303,
    "no_tumor": 139,
    "pituitary": 300
}
EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp"}
DATASET_ROOT.mkdir(exist_ok=True)

# Fix previously extracted Windows-style paths
fixed = 0
for src in list(DATASET_ROOT.iterdir()):
    if not src.is_file() or "\\" not in src.name:
        continue
    parts = src.name.replace("\\", "/").split("/")
    if len(parts) < 2:
        continue
    label, filename = parts[-2], parts[-1]
    if label not in EXPECTED:
        continue
    if Path(filename).suffix.lower() not in EXTENSIONS:
```

```
        continue
    dest = DATASET_ROOT / label / filename
    dest.parent.mkdir(exist_ok=True)
    if dest.exists():
        raise FileExistsError(f"Duplicate image: {dest}")
    src.rename(dest)
    fixed += 1

def count_images(label):
    folder = DATASET_ROOT / label
    if not folder.is_dir():
        return 0
    return sum(
        1 for p in folder.rglob("*")
        if p.is_file() and p.suffix.lower() in EXTENSIONS
    )

# Extract missing files directly into correct folders
if any(count_images(c) != n for c,n in EXPECTED.items()):
    assert ZIP_PATH.is_file(), "ZIP file not found!"
    with ZipFile(ZIP_PATH, "r") as z:
        for member in z.infolist():
            parts = member.filename.replace("\\", "/").split("/")
            if len(parts) < 2:
                continue
            label, filename = parts[-2], parts[-1]
            if label not in EXPECTED:
                continue
```

```
            if Path(filename).suffix.lower() not in EXTENSIONS:
                continue
            dest = DATASET_ROOT / label / filename
            dest.parent.mkdir(exist_ok=True)
            if not dest.exists():
                with z.open(member) as source:
                    with open(dest, "wb") as output:
                        shutil.copyfileobj(source, output)

print("=== LAB 06 - MRI TEST DATASET ===")
print("Dataset path:", DATASET_ROOT.resolve())
print("Windows paths fixed:", fixed)
print()
total = 0
for label, expected in EXPECTED.items():
    count = count_images(label)
    total += count
    print(f"{label}: {count} images")
    assert count == expected, f"Count mismatch: {label}"
print("\nTotal:", total)
assert total == 995, "Dataset count mismatch!"
print("PASS: All 995 MRI test images are ready!")
```

Step 3. Chương trình đánh giá ARM và ARM + FPGA

Ô code thứ hai mở Overlay và hai Interpreter một lần, chuyển từng ảnh về 96 × 96 grayscale INT8, warm-up trên năm ảnh mẫu rồi chạy đủ 995 ảnh bằng hai phương án. Vòng lặp luân phiên thứ tự thực thi để hạn chế thiên lệch thời gian, so sánh toàn bộ đầu ra bằng np.array_equal và lưu kết quả từng ảnh vào CSV.

Thời gian được đo bằng time.perf_counter(), gồm ba thành phần: ARM thuần, FPGA + DMA và ARM tail; thời gian Hybrid là toàn bộ chuỗi FPGA + ARM tail. Các phép đo **không gồm** đọc file ảnh, resize, khởi tạo Interpreter hoặc nạp Overlay.

```
# CELL 2 - EVALUATE ARM AND ARM + FPGA
import time
import csv
from pathlib import Path
import numpy as np
from PIL import Image
from pynq import Overlay, allocate
from tflite_runtime.interpreter import Interpreter

LABELS = ["glioma", "meningioma", "no_tumor", "pituitary"]
EXPECTED = [253, 303, 139, 300]
DATA = Path("test_data")
ASSETS = Path("../Lab_05")
EXT = {".jpg", ".jpeg", ".png", ".bmp"}

# Collect the 995 labeled samples
samples = []
for i, name in enumerate(LABELS):
    paths = sorted(
        p for p in (DATA / name).rglob("*")
        if p.is_file() and p.suffix.lower() in EXT
    )
    assert len(paths) == EXPECTED[i], name
    samples.extend((p,i) for p in paths)
assert len(samples) == 995

for filename in [
```

```
    "conv1_accel.bit", "conv1_accel.hwh",
    "tiny_cnn_v5_int8.tflite",
    "tiny_cnn_v5_tail_int8.tflite"
]:
    assert (ASSETS / filename).is_file(), filename

# Load models and FPGA once
ol = Overlay(str((ASSETS / "conv1_accel.bit").resolve()))
dma, conv1 = ol.axi_dma_0, ol.conv1_hls_0
full = Interpreter(
    model_path=str(ASSETS / "tiny_cnn_v5_int8.tflite"),
    num_threads=1
)
tail = Interpreter(
    model_path=str(ASSETS / "tiny_cnn_v5_tail_int8.tflite"),
    num_threads=1
)
full.allocate_tensors()
tail.allocate_tensors()
fi = full.get_input_details()[0]["index"]
fo = full.get_output_details()[0]["index"]
ti = tail.get_input_details()[0]["index"]
to = tail.get_output_details()[0]["index"]
assert tuple(tail.get_input_details()[0]["shape"]) == (1,47,47,8)
ibuf = allocate(shape=(9216,),dtype=np.int8)
obuf = allocate(shape=(47,47,8),dtype=np.int8)
```

```
# Preprocess a 96 x 96 grayscale INT8 image
def prepare(path):
    with Image.open(path) as img:
        img = img.convert("L").resize((96,96),Image.BILINEAR)
        pixels = np.asarray(img,dtype=np.int16)
    return (pixels-128).astype(np.int8).reshape(1,96,96,1)

def run_arm(x):
    t0 = time.perf_counter()
    full.set_tensor(fi,x)
    full.invoke()
    result = full.get_tensor(fo).copy()
    return result,(time.perf_counter()-t0)*1000

def run_hybrid(x):
    t0 = time.perf_counter()
    ibuf[:] = x.reshape(-1)
    ibuf.flush()
    dma.recvchannel.transfer(obuf)
    conv1.write(0x00,1)
    dma.sendchannel.transfer(ibuf)
    deadline = time.monotonic()+3.0
    while True:
        tx = dma.mmio.read(0x04)
        rx = dma.mmio.read(0x34)
        if (tx | rx) & 0x70:
            raise RuntimeError(f"DMA error: {hex(tx)}, {hex(rx)}")
```

```
        if (tx & 2) and (rx & 2):
            break
        if time.monotonic() >= deadline:
            raise TimeoutError(
                f"DMA timeout: {hex(tx)}, {hex(rx)}"
            )
    obuf.invalidate()
    t1 = time.perf_counter()
    pool1 = np.asarray(obuf).reshape(1,47,47,8)
    tail.set_tensor(ti,pool1)
    tail.invoke()
    result = tail.get_tensor(to).copy()
    t2 = time.perf_counter()
    return (result,(t1-t0)*1000,
            (t2-t1)*1000,(t2-t0)*1000)

print("Warming up...")
for p,_ in samples[::200][:5]:
    x = prepare(p)
    a,_ = run_arm(x)
    b,_,_,_ = run_hybrid(x)
    assert np.array_equal(a,b)

print("Evaluating 995 MRI images...")
records = []
cm_arm = np.zeros((4,4),dtype=np.int64)
cm_hybrid = np.zeros((4,4),dtype=np.int64)
```

```
for n,(path,label) in enumerate(samples,1):
    x = prepare(path)
    if n % 2:
        a,arm_ms = run_arm(x)
        b,fpga_ms,tail_ms,hybrid_ms = run_hybrid(x)
    else:
        b,fpga_ms,tail_ms,hybrid_ms = run_hybrid(x)
        a,arm_ms = run_arm(x)
    arm_pred = int(np.argmax(a))
    hybrid_pred = int(np.argmax(b))
    match = bool(np.array_equal(a,b))
    cm_arm[label,arm_pred] += 1
    cm_hybrid[label,hybrid_pred] += 1
    records.append([
        str(path),LABELS[label],LABELS[arm_pred],
        LABELS[hybrid_pred],int(match),arm_ms,
        fpga_ms,tail_ms,hybrid_ms
    ])
    if n % 100 == 0 or n == 995:
        print(f"Completed {n}/995 images")

header = [
    "image","true_label","arm_prediction",
    "hybrid_prediction","outputs_match","arm_ms",
    "fpga_dma_ms","tail_ms","hybrid_ms"
]
with open("lab06_predictions.csv","w",newline="") as f:
```

```
    w = csv.writer(f)
    w.writerow(header)
    w.writerows(records)

def metrics(cm):
    tp = np.diag(cm).astype(float)
    precision = tp/np.maximum(cm.sum(axis=0),1)
    recall = tp/np.maximum(cm.sum(axis=1),1)
    f1 = 2*precision*recall/np.maximum(precision+recall,1e-12)
    accuracy = tp.sum()/cm.sum()
    return accuracy,precision,recall,f1

def save_matrix(name,cm):
    with open(name,"w",newline="") as f:
        w = csv.writer(f)
        w.writerow(["Actual / Predicted"]+LABELS)
        for label,row in zip(LABELS,cm):
            w.writerow([label]+row.tolist())

save_matrix("lab06_confusion_arm.csv",cm_arm)
save_matrix("lab06_confusion_hybrid.csv",cm_hybrid)
acc_a,prec_a,rec_a,f1_a = metrics(cm_arm)
acc_h,prec_h,rec_h,f1_h = metrics(cm_hybrid)
times = np.asarray([r[5:] for r in records],dtype=float)
arm_med,fpga_med,tail_med,hybrid_med = np.median(times,axis=0)
exact_matches = sum(r[4] for r in records)
```

```
lines = [
    "=== LAB 06 - 995 MRI TEST IMAGES ===",
    f"Total images: {len(records)}",
    f"Identical ARM/Hybrid outputs: {exact_matches}/995",
    f"ARM accuracy: {acc_a*100:.2f}%",
    f"Hybrid accuracy: {acc_h*100:.2f}%",
    f"ARM macro F1: {np.mean(f1_a):.4f}",
    f"Hybrid macro F1: {np.mean(f1_h):.4f}","",
    "=== MEDIAN INFERENCE TIME ===",
    f"ARM only: {arm_med:.3f} ms",
    f"FPGA + DMA: {fpga_med:.3f} ms",
    f"ARM tail: {tail_med:.3f} ms",
    f"ARM + FPGA: {hybrid_med:.3f} ms",
    f"Speedup (ratio of medians): {arm_med/hybrid_med:.3f}x",
    f"Speedup (total inference time): "
    f"{times[:,0].sum()/times[:,3].sum():.3f}x", "",
    "=== CONFUSION MATRIX - ARM ===",str(cm_arm),"",
    "=== CONFUSION MATRIX - HYBRID ===",str(cm_hybrid),"",
    "=== PER-CLASS PRECISION / RECALL / F1 (HYBRID) ==="
]
for i,name in enumerate(LABELS):
    lines.append(
        f"{name}: P={prec_h[i]:.4f} "
        f"R={rec_h[i]:.4f} F1={f1_h[i]:.4f}"
    )
summary = "\n".join(lines)
print("\n"+summary)
```

```
Path("lab06_summary.txt").write_text(summary,encoding="utf-8")
ibuf.freebuffer()
obuf.freebuffer()
print("\nPASS: Evaluation completed.")
print("Saved CSV and summary files.")
```

Step 4. Kết quả độ chính xác và ma trận nhầm lẫn

Kết quả chương trình trên PYNQ-Z2 cho **995/995 tensor INT8 đầu ra ARM và Hybrid trùng khớp**. Với thứ tự nhãn glioma, meningioma, no_tumor, pituitary như trong Notebook, cả hai phương án có 922 ảnh dự đoán đúng và 73 ảnh sai: **Accuracy 92,66%, Macro F1 = 0,9194**.

Nhãn thật / Dự đoán	Glioma	Mening.	No tumor	Pituitary
Glioma	244	7	0	2
Meningioma	17	264	12	10
No tumor	3	13	117	6
Pituitary	1	2	0	297

Bảng 6.1. Confusion Matrix giống nhau của ARM thuần và ARM + FPGA.

Lớp MRI	Precision	Recall	F1-score
Glioma	0,9208	0,9644	0,9421
Meningioma	0,9231	0,8713	0,8964
No tumor	0,9070	0,8417	0,8731
Pituitary	0,9429	0,9900	0,9659

Bảng 6.2. Precision, Recall và F1-score theo từng lớp MRI.

Pituitary có Recall cao nhất (297/300 ảnh, 99,00%), trong khi No tumor thấp nhất (117/139 ảnh, 84,17%). Hai cấu hình có cùng Confusion Matrix, chứng minh FPGA không làm thay đổi quyết định phân loại trong tập Test này.

Step 5. Kết quả đo thời gian suy luận

Thành phần	Thời gian trung vị
ARM thuần	23,644 ms
FPGA + DMA	1,452 ms
ARM tail	16,431 ms
ARM + FPGA	**17,855 ms**
Speedup	**1,324×**

```
=== LAB 06 - 995 MRI TEST IMAGES ===
Total images: 995
Identical ARM/Hybrid outputs: 995/995
ARM accuracy: 92.66%
Hybrid accuracy: 92.66%
ARM macro F1: 0.9194
Hybrid macro F1: 0.9194

=== MEDIAN INFERENCE TIME ===
ARM only: 23.644 ms
FPGA + DMA: 1.452 ms
ARM tail: 16.431 ms
ARM + FPGA: 17.855 ms
Speedup (ratio of medians): 1.324x
Speedup (total inference time): 1.324x
```

Hình 6.1. Kết quả đánh giá tự động 995 ảnh MRI trên PYNQ-Z2.

Từ hai trung vị, thời gian Hybrid giảm **5,789 ms** so với ARM thuần và hệ số tăng tốc đạt **1,324×**. Tỷ số tổng thời gian trên 995 ảnh cũng xấp xỉ 1,324×. Trung vị của tổng thời gian Hybrid có thể khác tổng các trung vị thành phần, do mỗi dãy thời gian được tính trung vị riêng biệt.

Step 6. Đối chiếu kết quả với Edge Impulse

Ở Lab 03, mô hình Quantized INT8 đạt **93,07% Test Accuracy** trên Edge Impulse; kết quả PYNQ-Z2 trong Lab 06 là **92,66%**, chênh **0,41 điểm phần trăm**. Edge Impulse có cột **UNCERTAIN**, còn Notebook chọn lớp bằng argmax; hai Confusion Matrix vì thế không hoàn toàn cùng cách thống kê.



Hình 6.2. Confusion Matrix của mô hình Quantized INT8 trên Edge Impulse.

Edge Impulse đặt ảnh 96 × 96, chế độ Fit longest axis và Grayscale; Notebook sử dụng Pillow convert("L") kết hợp nội suy bilinear rồi chuyển INT8. Hai pipeline chưa được xác minh giống hệt theo từng pixel. Recall No tumor trên PYNQ-Z2 là 84,17% trong khi giao diện Edge Impulse hiển thị 99,3%; báo cáo ghi nhận chênh lệch này mà chưa quy kết nguyên nhân.

Step 7. Kết luận Lab 06

Lab 06 đã hoàn thành kiểm thử **995 ảnh MRI Test** với kết quả ARM và Hybrid khớp 995/995 tensor đầu ra INT8. Theo thứ tự nhãn đang sử dụng, hệ thống đạt **92,66% Accuracy**, **0,9194 Macro F1** và cấu hình kết hợp nhanh hơn **1,324×** về thời gian suy luận trung vị. Đây là bằng chứng thực nghiệm về tính tương đương kết quả và hiệu quả tăng tốc Conv1 trên PYNQ-Z2; phép đo không bao gồm đọc và tiền xử lý ảnh.

Các tệp lưu kết quả gồm **lab06_predictions.csv**, **lab06_confusion_arm.csv**, **lab06_confusion_hybrid.csv** và **lab06_summary.txt**. Số liệu trong báo cáo dựa trên dữ liệu đã đo thực tế, không phải mô phỏng hoặc ước lượng.